

Read-Only Memory (ROM) is a type of **non-volatile memory** that stores permanent, unchanging instructions—known as **firmware**—required to start up and operate a device. The primary types of ROM are classified based on how they are programmed and, importantly, how or if they can be erased and reprogrammed.

Types of Read-Only Memory (ROM)

Type	Full Form	Programming Method	Erase Method
MROM	Mask ROM	Hard-wired during the manufacturing process.	Cannot be erased or reprogrammed (read-only).
PROM	Programmable ROM	Programmed once by the user (not the manufacturer) using a special device (a PROM programmer).	Cannot be erased or reprogrammed after the initial write.
EPROM	Erasable PROM	Programmed electrically by the user.	Erased by exposure to strong ultraviolet (UV) light through a quartz window on the chip. Can be reprogrammed afterward.
EEPROM	Electrically Erasable PROM	Programmed and erased by the user using electrical signals ; no need for UV light or chip removal.	Erased byte-by-byte electrically. Can be rewritten thousands of times.

Explanation of Key Types

- PROM (Programmable Read-Only Memory):**
 - This chip starts as blank memory. The user (not the chip manufacturer) can program it **once** by "burning" fusible links or anti-fuses within the chip using a high-voltage current. Once a link is burned, it's permanent, meaning PROM is a **One-Time Programmable (OTP)** device and cannot be erased.
- EPROM (Erasable Programmable Read-Only Memory):**
 - It is an improvement over PROM as it can be **reprogrammed**. Data is erased by removing the chip from the device and exposing it to intense **ultraviolet light** for a specified period (e.g., up to 40 minutes). This process clears the entire chip, which must then be reprogrammed completely.
- EEPROM (Electrically Erasable Programmable Read-Only Memory):**
 - This is the most flexible type (excluding Flash Memory, a modern variant). It can be erased and reprogrammed **electrically** without removing the chip from the

circuit. The key advantage is its ability to erase and rewrite data **byte by byte** (individual data locations) rather than clearing the entire chip.

4. **MROM (Mask Read-Only Memory):**

- o This is the earliest and oldest type of ROM. The data is physically embedded into the integrated circuit during the manufacturing process by creating a photographic "mask." It is the most inexpensive for large volumes but has the major drawback of being **impossible to change** after fabrication.

WMFC

The acronym **WMFC** stands for **Wait for Memory Function Complete**. It is a crucial **control signal** used in the Control Unit of the Central Processing Unit (CPU) during memory access operations.

Why the WMFC Step is Needed

The WMFC step is required because the **CPU** and the **Main Memory (RAM)** operate asynchronously and at significantly different speeds.

1. **Speed Mismatch (Asynchronicity):** The CPU is a very high-speed component that operates based on its internal clock cycles (e.g., in nanoseconds). Main memory (RAM) is a slower component that requires a much longer duration (many CPU clock cycles) to complete a Read or Write operation.
2. **Coordination Mechanism:** When the CPU initiates a memory operation (like a READ for an instruction or data), it issues a command to the memory controller and then asserts the **WMFC** signal.
3. **Pausing the CPU:** Asserting WMFC effectively tells the Control Unit to **pause** its internal step counter. The CPU is forced to wait in the current control step (often T2 in the fetch cycle) and cannot proceed to the next step.
4. **Completion Signal (MFC):** The memory controller, after completing the requested operation (fetching the data for a Read or confirming the data has been stored for a Write), generates a reciprocal signal called **Memory Function Complete (MFC)**.
5. **Resuming CPU Operation:** The MFC signal deactivates the WMFC state, allowing the Control Unit to proceed to the next control step (e.g., moving the data from the Memory Data Register (MDR) to the Instruction Register (IR) or a general-purpose register).

In essence, **WMFC prevents the fast CPU from trying to use data that hasn't arrived yet (Read) or overwriting a location before the prior data has been fully saved (Write)**, ensuring proper synchronization between the processor and the much slower memory subsystem.

Cache replacement algorithms are essential in **associative memory mapping** (Fully Associative or Set-Associative) because multiple main memory blocks can map to the same set or entire cache. When a **cache miss** occurs and the corresponding set is full, one of the existing blocks must be evicted to make space for the new block. The replacement algorithm determines *which* block is evicted.

Cache Replacement Algorithms

1. FIFO (First-In, First-Out)

- **Principle:** Evicts the block that has been in the cache (or the specific set) for the **longest time**, regardless of how recently or frequently it was used.
- **Mechanism:** It works like a **queue**. When a block is inserted, it goes to the back; when a block needs to be replaced, the one at the front (the oldest) is chosen.
- **Implementation:** Simple to implement, typically using a counter or timestamp for each block upon entry.
- **Disadvantage:** It performs poorly if a frequently used block happens to be one of the first ones loaded.

2. LRU (Least Recently Used)

- **Principle:** Evicts the block that has been **least recently accessed**. The underlying assumption is that a block used recently is more likely to be used again soon (**Temporal Locality**).
- **Mechanism:** Every time a block is accessed (hit or miss), its **usage time** is updated. When replacement is needed, the block with the oldest usage time is chosen.
- **Implementation:** More complex than FIFO, requiring hardware or software to track the **recency** of all blocks in a set (e.g., using counters, timestamps, or a priority list/linked list structure).
- **Advantage:** Generally achieves a much higher **hit ratio** than FIFO.

Cache Replacement Algorithms in Associative Mapping

Cache replacement algorithms are essential in **associative memory mapping** (Set-Associative or Fully Associative). When a **cache miss** occurs and the corresponding cache set is full, the algorithm determines which existing block to evict to make room for the new data.

1. 🏆 FIFO (First-In, First-Out)

- **Principle:** Evicts the block that has been resident in the cache (or the specific set) for the **longest time**, regardless of how frequently or recently it was used.

- **Mechanism:** It operates like a queue. The block that was loaded **first** is the one that is replaced **first**.
 - **Implementation:** Simple to implement, typically by maintaining a **queue structure** or an **insertion time counter** for each block.
 - **Disadvantage:** It fails to consider **temporal locality**. A frequently used block might be evicted simply because it was one of the first blocks loaded, leading to a potential decrease in the hit ratio.
-

2. 🕒 LRU (Least Recently Used)

- **Principle:** Evicts the block that has been **least recently accessed**. It is based on the strong principle of **temporal locality**, assuming that a block used recently is more likely to be used again soon.
- **Mechanism:** Every time a block is accessed (hit or miss), its **usage recency** is updated. When replacement is needed, the block with the oldest usage time is chosen.
- **Advantage:** It generally achieves a significantly **higher hit ratio** than FIFO because it protects actively used blocks from being replaced.
- **Disadvantage:** It is **complex and expensive** to implement in hardware, as it requires tracking the precise access order of all blocks within a set (e.g., using counters or a complex list structure).

LRU CACHE REPLACEMENT ALGORITHM

Cache replacement algorithms are efficiently designed to replace the cache when the space is full. The **Least Recently Used (LRU)** is one of those algorithms. As the name suggests when the cache memory is full, LRU picks the data that is least recently used and **removes** it in order to make space for the new data. The priority of the data in the cache changes according to the need of that data i.e. if some data is fetched or updated recently then the priority of that data would be changed and assigned to the **highest priority**, and the priority of the data **decreases** if it remains unused operations after operations.

Operations on LRU Cache:

- **LRUCache (Capacity c):** Initialize LRU cache with positive size capacity **c**.
- **get (key) :** Returns the value of key ' **k**' if it is present in the cache otherwise it returns -1. Also updates the priority of data in the LRU cache.
- **put (key, value):** Update the value of the key if that key exists, Otherwise, add a **key-value** pair to the cache. If the number of keys exceeds the capacity of the LRU cache then dismiss the least recently used key.

Working of LRU Cache:

Let's suppose we have an LRU cache of **capacity 3**, and we would like to perform the following operations:

- **put (key=1, value=A)** into the cache
- **put (key=2, value=B)** into the cache
- **put (key=3, value=C)** into the cache
- **get (key=2)** from the cache
- **get (key=4)** from the cache
- **put (key=4, value=D)** into the cache
- **put (key=3, value=E)** into the cache
- **get (key=4)** from the cache
- **put (key=1, value=A)** into the cache

LRU(Least Recently Used)

🔗 This algorithm replaces the cache memory block by main memory block, the block which is least recently used.

🔗 Let memory reference is given :- 7, 0, 1, 2, 0, 3, 5, 4, 2, 3, 0

🔗 Number of Cache block=4

3				2	2	2	2	4	4	4	4
2			1	1	1	1	5	5	5	5	0
1		0	0	0	0	0	0	0	2	2	2
0	7	7	7	7	7	3	3	3	3	3	3
	7 miss	0 miss	1 miss	2 miss	0 hit	3 miss	5 miss	4 miss	2 miss	3 hit	0 miss

[Cache replacement policies,cache miss,writingtechniques | PDF](#)

Real Time Embedded Systems

Real time embedded systems can be defined as;

- "The embedded systems which respond to real time situation with the help of its embedded software and hardware, within the specified time constraints are called real time embedded systems."

Characteristics of a Real-Time Embedded System

Real time embedded systems must have the following characteristics;

- A real-time embedded system always responds in the same manner to a certain situation, it is not allowed to deviate from its normal designated output. An air-conditioner is not allowed to throw hot air in summers.
- A deadline is crucial to the working of an embedded system, a missed deadline can cost lives and finances.
- In case of any malfunctioning, the system failure can cause havoc, what would happen if the pacemaker can't maintain the heartbeat, patient would eventually die!
- It is the most important characteristic of all, the real-time embedded system must be swift enough to respond to the changing external environment with immediate effect.

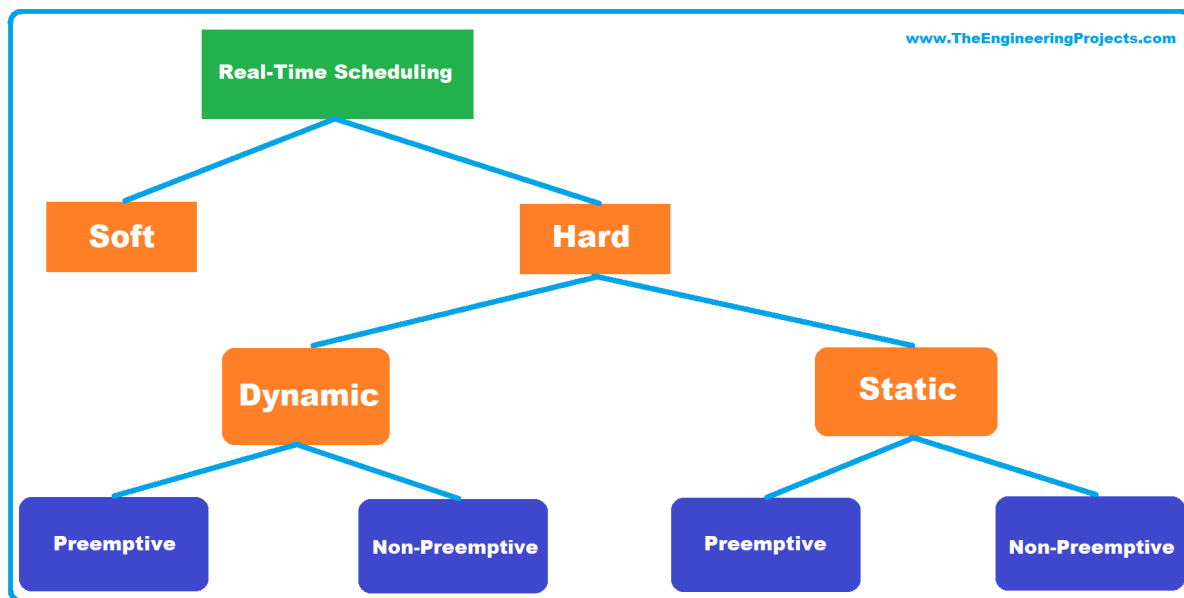
Components of Real Time Embedded Systems

We have already discussed the embedded system components in detail within our previous article, for a quick overview , let's revise the basic things;

- The hardware parts include a microcontroller or a microprocessor, Input and Output ports, sensors , actuators , relays, power supply or batteries and several other peripheral parts according to the design and function of the embedded system.

- Real time embedded systems have embedded software which directs the system for performing designated tasks.
- For a real time embedded system, embedded operating systems software must have the critical feature of task scheduling, because we need a system which sticks to the deadline and performs the task within that limited time range. Let's have a look, how task scheduling is done;

Task Scheduling



Examples of Real time embedded systems

1. **Automotive Systems:** Embedded systems are used in vehicles such as airbags, anti-lock braking systems (ABS), and GPS trackers, which require quick responses to ensure safety and efficiency.
2. **Medical Devices:** Pacemakers and other medical equipment operate in real-time to maintain heart function and deliver treatments precisely.
3. **Industrial Automation:** Robots in factories perform tasks in real-time, ensuring precision and efficiency in manufacturing processes.
<https://www.bing.com/ck/a?!&&p=187b007309c0ce46b32588b416e9d321c54e5f5bc835bd77e7cf5c1d5ca790c6JmltdHM9MTc2MzMzMzYwMA&ptn=3&ver=2&hsh=4&fclid=1383ad97-53f0-6f13-1b04-bbcc528f6e33&u=a1aHR0cHM6Ly93d3cudGhlZW5naW5lZXJpbmdwcm9qZWNOcy5jb20v>

4. **Smart Home Devices:** Systems like smart locks, security cameras, and lighting control operate in real-time to enhance home automation and security

An interrupt is a signal from a device attached to a computer or from a program within the computer that requires the operating system to stop and figure out what to do next.

Whenever an interrupt occurs, it causes the CPU to stop executing the current program. Then comes the control to interrupt the handler or interrupt service routine.

- **Step 1** – When an interrupt occurs let assume processor is executing i'th instruction and program counter will point to the next instruction (i+1)th.
- **Step 2** – When an interrupt occurs the program value is stored on the process stack and the program counter is loaded with the address of interrupt service routine.
- **Step 3** – Once the interrupt service routine is completed the address on the process stack is popped and placed back in the program counter.
- **Step 4** – Now, it executes the resume for (i+1)th line.

Multiple interrupt is an interrupt event which can occur while the processor is handling a previous interrupt. For example – A program receiving data from a communication line and printing result there is a possibility for communication interrupt to occur while printer interrupt being processed.

There are two ways by which processor can handle interrupts

- **Disable interrupt** – Processor will ignore further interrupts while processing one interrupt. Interrupts remain pending and are checked after the first interrupt has been handled. In this process interrupts are handled in sequence
- **Define priorities** – In this method low priority interrupts can be interrupted by higher priority interrupts. Here, high priority interrupt will be handled then processor returns to previous interrupt on which it was earlier working.

There are two types of interrupts –

- **Hardware interrupts** – Interrupt signal generated from external devices.
- **Software interrupts** – Interrupt signal generated from internal devices.

We know that the instruction cycle consists of fetch, decode, execute and read/write functions.

After every instruction cycle, the processor will check for interrupts to be processed. If no interrupt is present in the system it will go for the next instruction cycle which is given by the instruction register.

If there is an interrupt present then it will trigger the interrupt handler, the handler will stop the present instruction which is processing and save its configuration in a register and load the program counter of the interrupt from a location which is given by the interrupt table.

After processing the interrupt the processor interrupt handler will load the instruction and its configuration from the saved register, the process will start its processing where it's left. This is also called context switching.